

# Construction Element Detection for Site Progress Verification

A cloud-only YOLOv8 pipeline for automated progress evidence

# Progress evidence is captured, then lost

- **Latency.** Two to seven days between the photograph and the progress record. Decisions get made on stale data.
- **Inconsistency.** Two engineers describe the same floor differently. No controlled vocabulary links what is visible to a structured record.
- **Loss of information.** The photograph is archived. What is *in* it — steelbar placed, blockwork started, conduit run — is never captured as data, so it cannot enter a CDE or be audited.

In the Gulf, progress reporting on large programmes is still photograph-plus-narrative, with a two-to-seven-day lag between capture and record. Government BIM mandates and ISO 19650 information-delivery requirements are raising the bar on structured, auditable progress evidence. A detection record — image, time, zone, class, confidence — is structured information that can enter a CDE. A JPEG in a folder is not.

# Five classes, one detector, zero local installs

Detect construction elements and plant in site photographs, then use presence as a progress and site-condition signal.

| Class     | Signal it carries                            |
|-----------|----------------------------------------------|
| steelbar  | Reinforcement activity — structural progress |
| brick     | Masonry and blockwork — envelope progress    |
| pvcpipe   | Services installation — MEP first fix        |
| scaffold  | Temporary works standing — zone not released |
| excavator | Plant present — earthworks active            |

Pipeline

- Roboflow — fork, extend, annotate
- Colab — YOLOv8, 30 epochs, T4
- Validate — P / R / mAP, per class
- Evidence — validation + unseen images
- Handover — repo, governance, licence

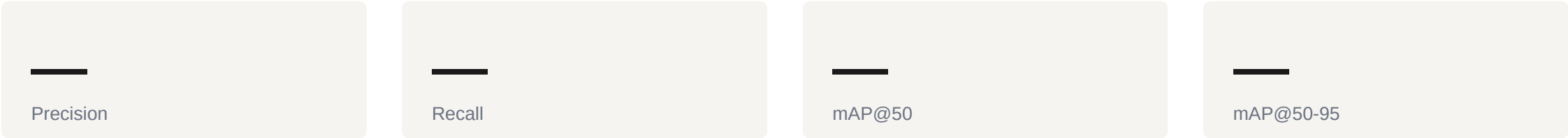
# A public base, re-split and extended

Forked **seungyeon/construction-site-km7bh** (700 images, CC BY 4.0) into our own Roboflow workspace, re-split it 80/20, and added our own images including deliberate boundary cases — the things that look like a class and are not.

| Boundary case | Looks like | Actually is                       | Rule it tests                  |
|---------------|------------|-----------------------------------|--------------------------------|
| _____         | steelbar   | Loose scaffold tube on the ground | Belongs to NEITHER class       |
| _____         | scaffold   | Ladder, handrail, hoarding frame  | Erected access, not any frame  |
| _____         | brick      | Stone cladding, paving block      | Material versus form           |
| _____         | pvcpipe    | Metal conduit under dust          | Material when colour fails     |
| _____         | excavator  | Wheeled loader, dozer             | Machine type, not yellow plant |

**Why boundary cases matter.** A class is defined by its edge, not its centre. Without written rules and images that test them, annotation drifts between people and the model learns the drift.

# Validation performance



| Class     | P | R | mAP@50 | mAP@50-95 |
|-----------|---|---|--------|-----------|
| brick     | — | — | —      | —         |
| excavator | — | — | —      | —         |
| pvcpipe   | — | — | —      | —         |
| scaffold  | — | — | —      | —         |
| steelbar  | — | — | —      | —         |

**Success criteria.** S1 overall mAP@50 ≥ 0.50 — \_\_\_\_\_. S2 steelbar recall ≥ 0.50 — \_\_\_\_\_. S3 brick recall ≥ 0.40 — \_\_\_\_\_. Targets were set before the run and are reported whether met or missed.

# Three things the numbers say

- 1 Against success criterion S1 ( $\text{mAP@50} \geq 0.50$ ) the run scored \_\_\_\_\_. Replace this with one sentence on what that means for the triage use case.
- 2 \_\_\_\_\_ performs best and \_\_\_\_\_ worst. The gap is explained by \_\_\_\_\_ (instance count, object scale, or annotation consistency).
- 3 The dominant error mode is \_\_\_\_\_. It matters because \_\_\_\_\_.

# The two errors are not symmetrical

## False negative

Severity: High

Steelbar is placed, the model misses it. No record is created. Nobody investigates, because nobody knows there is anything to investigate. **The error is silent.**

## False positive

Severity: Moderate

The model reports steelbar that is not there. The next site walk contradicts it. **The error is loud and self-correcting** — but repeated, it produces alert fatigue.

## One model, two thresholds

Progress logging runs at a **low** confidence threshold, favouring recall because a miss is silent. Site-condition alerts run at a **high** one, favouring precision because false alarms train people to ignore the system. A single threshold would be wrong for one of them.

Three false positives and three false negatives are documented with filenames and hypotheses in *docs/error\_analysis.md*, each tied to a dataset gap rather than to model capacity — and each paired with a concrete improvement and the metric it should move.

# When not to use this

- No safety function. The model detects no people, no PPE and no hazards. Treating it as a safety system would reduce site safety, not improve it.
- Presence only. It cannot measure quantity, spacing, cover or conformance, and its detection counts must never enter a payment application.
- Validated scope is daytime site photography at working range. Night, dust, rain, drone and enclosed-space imagery are untested.
- Outputs are uncalibrated. A 0.9 confidence is not a 90 percent probability, and no detection is contractual evidence.

## Rights

CODE

**MIT**

DATASET

**CC BY 4.0, not owned**

WEIGHTS

**AGPL-3.0 inherited**

PEOPLE DETECTED

**None, by design**

Commercial deployment of YOLOv8-derived weights requires AGPL-3.0 compliance or an Ultralytics licence. This is the most commonly missed obligation in YOLO projects.